

BOOTCAMP KOFFIEDEV · SUPPORT DE COURS

Jour 1. Réseaux et développement web.

Support de cours à lire, annoter et garder. Imprime-le ou suis-le à l'écran. Les corrigés se font ensemble, pendant la séance.

Gratuit

Jour 1 · 2 h

Débutant bienvenu

En visio · 12 h GMT

AUJOURD'HUI TU REPARS AVEC

La notion client / serveur, le nom de domaine, les protocoles d'envoi d'une page, et une vitrine HTML avec un lien WhatsApp.

Garde ce support toute la semaine. Il servira encore.

Prépare ça, rien de plus

- ☐ Un ordinateur (le téléphone seul ne suffit pas pour coder).
- ☐ Chrome, Edge ou Firefox.
- ☐ Un dossier vide sur le Bureau, nommé `vitrine`.
- ☐ WhatsApp, pour tester ton lien à la fin.

PAS D'ÉDITEUR INSTALLÉ ?

On peut tout faire dans le Bloc-notes, ou sur <https://codepen.io/pen> (onglets HTML et CSS).

OBJECTIF DU JOUR

Comprendre comment une page est demandée et envoyée sur Internet, puis en créer une. Pas besoin d'avoir déjà codé.

Les trois temps du jour

	ON TRAVAILLE	TU REPARS AVEC
1	Client / serveur, domaine, protocoles d'envoi	Le voyage d'une page, dans l'ordre
2	HTML et un peu de CSS	Les balises pour une vraie page
3	Mini-projet vitrine	Ton fichier <code>index.html</code> + lien WhatsApp

1. Client et serveur

Sur Internet, presque tout est un dialogue entre deux rôles. L'un **demande**, l'autre **répond**. C'est le modèle **client / serveur**.

Le client

Le programme qui **initie** la demande. Exemples : Chrome, Firefox, l'app WhatsApp, ton téléphone. Il n'a pas la page en réserve : il la réclame.

Le serveur

Le programme (sur une machine souvent allumée en continu) qui **attend** les demandes et envoie la réponse : fichier HTML, image, vidéo, message.

Le dialogue, à chaque fois

1. Le client ouvre une connexion vers le serveur.
2. Il envoie une **requête** : « donne-moi telle ressource ».
3. Le serveur traite, puis envoie une **réponse** (succès ou erreur).
4. Le client reçoit et affiche (ou enregistre) le résultat.

Analogie resto : tu commandes (client), la cuisine prépare (serveur), le plateau arrive. Cuisine fermée → pas de réponse. Plat introuvable → 404 . Commande OK → 200 .

Points à bien verrouiller

- Le **client n'est pas « toi »** : c'est le logiciel qui demande (le navigateur).
- Le **serveur n'est pas « Internet »** : c'est une machine précise, avec une adresse.
- Une même machine peut être les deux : aujourd'hui, ton PC est serveur (fichier local) et client (Chrome).
- Un serveur peut répondre à des milliers de clients en même temps.
- Sans serveur joignable, le plus bel HTML du monde n'arrive nulle part.

SITUATION	CLIENT	SERVEUR
Tu ouvres YouTube	Chrome	Les machines de YouTube
Tu envoies « salut » sur WhatsApp	L'app sur ton téléphone	Les machines de WhatsApp
Tu ouvres <code>index.html</code> en local	Le navigateur	Ton propre disque / ordi

2. Nom de domaine

Une machine se trouve par une **adresse IP** (ex. 142.250.200.78). Quatre nombres entre 0 et 255, séparés par des points. C'est lisible pour une machine, pénible pour toi.

Le **nom de domaine** est le nom humain de ce lieu : google.com, whatsapp.com, wa.me. Comme le nom d'une boutique, au lieu du numéro de parcelle.

Anatomie d'un nom

Prends `www.koffiedev.com` (exemple) :

MORCEAU	EXEMPLE	RÔLE
Sous-domaine	www	Souvent le site web public. On peut avoir mail., api., etc.
Domaine	koffiedev	Le nom choisi, unique pour cette zone.
Extension (TLD)	.com	La « zone » : .com, .ci, .org, .edu ...

On lit un domaine **de droite à gauche** : d'abord la zone (.com), puis le nom, puis éventuellement `www` .

Le DNS : l'annuaire du domaine

Le **DNS** (Domain Name System) traduit un nom en IP. Sans lui, tu devrais retenir des chiffres pour chaque site.

1. Tu tapes `google.com` dans Chrome.
2. Le client demande à un serveur DNS : « quelle IP pour ce nom ? »
3. Le DNS répond, par exemple une adresse IPv4.
4. Ensuite seulement, Chrome va frapper à cette IP.

Analogie taxi : tu dis « emmène-moi chez Awa ». Le taxi ne connaît pas Awa. Il demande l'adresse (DNS), puis il y va (connexion vers l'IP).

SI LE DNS ÉCHOUÉ

Le navigateur affiche « introuvable » / « DNS_PROBE_FINISHED_NXDOMAIN ». Ce n'est pas encore le serveur web qui a parlé : le nom n'a pas d'IP connue.

À RETENIR

Le domaine est un **nom**. L'IP est une **adresse**. Le DNS est le **carnet d'adresses** qui relie les deux.

3. Protocoles d'envoi

Un **protocole** est un accord : comment on parle, dans quel ordre, avec quels mots. Sans protocole commun, le client et le serveur ne se comprennent pas, comme deux personnes sans langue partagée.

Pour envoyer une **page web**, trois protocoles se relaient. Tu n'as pas à les configurer aujourd'hui, mais tu dois savoir qui fait quoi.

PROTOCOLE	RÔLE, EN UNE PHRASE	QUAND TU LE VOIS
DNS	Demande l'IP qui correspond au nom de domaine.	Avant même d'ouvrir la page.
TCP	Ouvre une connexion fiable et envoie les données sans les perdre.	Invisible. C'est la route.
HTTP	La langue de la requête et de la réponse web (« donne-moi la page »).	Dans l'URL : <code>http://</code>
HTTPS	HTTP + chiffrement. Personne au milieu ne lit facilement.	Cadenas et <code>https://</code>

HTTP : la langue d'envoi de la page

HTTP = HyperText Transfer Protocol, le protocole de transfert de pages. Le client envoie une requête. Le plus courant aujourd'hui : **GET** (« donne-moi ce fichier »).

```
GET /index.html HTTP/1.1
Host: exemple.com
```

Le serveur répond avec un **code** et le contenu :

```
HTTP/1.1 200 OK
Content-Type: text/html

<!DOCTYPE html> ...
```

CODE	SIGNIFICATION
200	OK. Voici la ressource.
301 / 302	La page a déménagé. Va ailleurs.
403	Interdit. Tu n'as pas le droit.
404	Cette ressource n'existe pas chez moi.
500	Le serveur a un problème interne.

HTTPS : le même envoi, dans une enveloppe

HTTPS = HTTP + chiffrement (souvent via TLS). Le S veut dire *secure*. Le dialogue est le même (GET, 200, 404), mais un regard sur le wifi du café ne lit pas le contenu en clair.

- HTTP voyage souvent sur le **port 80**.
- HTTPS voyage sur le **port 443**.
- Un port, c'est la « porte » du serveur : l'IP est l'immeuble, le port est l'appartement.

RÈGLE DU BOOTCAMP

Pour une vitrine et un lien WhatsApp, on écrit toujours `https://`. On ne met jamais un mot de passe dans une page.

L'URL : protocole + domaine + ressource

L'URL assemble tout ce qu'on vient de voir. Exemple :

`https://wa.me/22890123456?text=Bonjour`

MORCEAU	EXEMPLE	RÔLE
Protocole d'envoi	<code>https://</code>	Comment on parle au serveur (ici : HTTP chiffré).
Nom de domaine	<code>wa.me</code>	Le nom. Le DNS le transforme en IP.
Chemin	<code>/22890123456</code>	Quelle ressource précise on GET.
Requête	<code>?text=Bonjour</code>	Un paramètre. Ici, le message prérempli.

Le voyage complet, dans l'ordre

1. Nom tapé

2. DNS → IP

3. TCP + HTTPS

4. GET / réponse

5. Page dessinée

1. Tu saisis le nom de domaine (ou tu cliques un lien).
2. Le DNS donne l'adresse IP du serveur.
3. TCP ouvre la connexion. HTTPS chiffre l'échange.
4. Le client (navigateur) envoie un GET. Le serveur répond 200 + HTML, puis CSS, images.
5. Le navigateur assemble et affiche. Toi tu vois « la page », pas l'IP.

AUJOURD'HUI

Ta vitrine vivra d'abord **sur ton ordinateur** (un fichier). Client et serveur sont la même machine. Le modèle reste le même. On n'héberge pas encore en ligne.

Exercices • réseaux

Écris tes réponses ici. On corrige ensemble pendant la séance.

EXERCICE 1 • CLIENT OU SERVEUR ?

Pour chaque situation : **client**, **serveur**, ou **les deux**.

1. Chrome, quand tu ouvres YouTube.
.....
2. L'ordinateur de YouTube qui envoie la vidéo.
.....
3. WhatsApp sur ton téléphone, quand tu envoies « salut ».
.....
4. La machine de WhatsApp qui transmet le message.
.....
5. Toi, quand tu ouvres ton fichier `index.html` dans le navigateur.
.....

EXERCICE 2 • LIRE UN NOM DE DOMAINE

Pour `www.koffiedev.com` :

1. Sous-domaine :
.....
2. Nom de domaine :
.....
3. Extension (TLD) :
.....
4. Qui transforme ce nom en IP ?
.....

EXERCICE 3 · PROTOCOLES D'ENVOI

Relie ou écris le bon protocole : **DNS** · **TCP** · **HTTP** · **HTTPS**.

1. Chiffrer l'échange pour que le wifi du café ne lise pas la page.

.....

2. Trouver l'IP de **wa.me** .

.....

3. Dire « GET /index.html » et recevoir le HTML.

.....

4. Transporter les données sans les perdre, une fois l'IP connue.

.....

EXERCICE 4 · DÉCORTIQUER L'URL

`https://wa.me/22890123456?text=Bonjour`

1. Protocole d'envoi :

.....

2. Nom de domaine :

.....

3. Chemin (le numéro) :

.....

4. Pourquoi il y a un **s** dans https ?

.....

5. Le client envoie plutôt un GET ou un « like » ?

.....

EXERCICE 5 · REMETTRE DANS L'ORDRE

Numérote de 1 à 6.

- ___ Le serveur répond **200** et envoie le HTML.
- ___ Tu tapes le nom de domaine dans Chrome.
- ___ HTTPS chiffre la connexion (port 443).
- ___ Le DNS donne l'adresse IP.
- ___ Le navigateur dessine la page à l'écran.
- ___ Le client envoie un GET au serveur.

Écrire une page HTML et un peu de CSS

Une fois la page *arrivée*, encore faut-il la *écrire*. **HTML** = la structure (murs et pièces). **CSS** = l'apparence (peinture et rideaux).

Une balise

```
<p>Ceci est un paragraphe.</p>
```

- `<p>` ouvre. `</p>` ferme. Le slash ferme.
- Ce qu'il y a au milieu s'affiche à l'écran.
- Écris les noms de balises en minuscules.

Le squelette de toute page

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <title>Ma vitrine</title>
    <style>
      /* le CSS viendra ici */
    </style>
  </head>
  <body>
    <h1>Mon prénom</h1>
    <p>Une phrase sur moi.</p>
  </body>
</html>
```

LIGNE	À QUOI ELLE SERT
<code><!DOCTYPE html></code>	Dit au navigateur : c'est du HTML moderne.
<code>lang="fr"</code>	Page en français.
<code><head></code>	Infos pour le navigateur, pas le texte principal.
<code>charset="UTF-8"</code>	Sans ça, les accents cassent.
<code><title></code>	Le nom de l'onglet.
<code><body></code>	Ce que tu vois.

Les balises du jour

BALISE	RÔLE	CHEZ TOI
<code><h1></code>	Titre principal (une fois)	Ton prénom
<code><h2></code>	Sous-titre	« Ce que je fais »
<code><p></code>	Paragraphe	2 ou 3 phrases
<code><a></code>	Lien	WhatsApp
<code></code> / <code></code>	Liste	3 envies de la semaine
<code></code>	Important	Un mot clé
<code></code>	Image	Photo, optionnelle

Le lien WhatsApp (une URL https)

```
<a href="https://wa.me/22890123456">Écrire sur WhatsApp</a>
```

- Protocole : `https` , envoi chiffré.
- Domaine : `wa.me` . Le DNS trouvera l'IP de WhatsApp.
- Numéro = indicatif pays + numéro, **sans + et sans espaces**.
- Togo 228 · Bénin 229 · Burkina 226 · Côte d'Ivoire 225 · Sénégal 221 · Mali 223
- Message prérempli : `?text=Bonjour` (un espace s'écrit `%20`).

Ton numéro, déjà au bon format :

Un peu de CSS

Une règle : **qui** (sélecteur), **quoi** (propriété), **combien** (valeur).

```
body {
  font-family: sans-serif;
  background: #efe6d6;
  color: #1a1612;
}

h1 {
  color: #c64520;
}

a {
  background: #167a39;
  color: white;
  text-decoration: none;
  padding: 12px 18px;
}
```

PROPRIÉTÉ	EFFET
color	Couleur du texte
background	Couleur de fond
font-family	La police
padding	Espace intérieur (bouton plus large)
text-decoration: none	Enlève le soulignement du lien

Exercices • HTML

EXERCICE 6 • RELIER

Relie chaque besoin à la bonne balise.

BESOIN	TA RÉPONSE
Nom de l'onglet	
Texte du bouton WhatsApp	
Une photo	
Liste de 3 compétences	
Phrase d'intro	

Choix : `title` • `a` • `img` • `ul/li` • `p`

EXERCICE 7 • PAGE CASSÉE

Cette page a **cinq** erreurs. Entoure-les, puis réécris la version correcte.

```
<html lang="fr">
  <head>
    <title>Ma vitrine
  </head>
  <body>
    <h1>Awa<h1>
    <p>Je découvre le web.
    <a href="https://wa.me/+228 90 12 34 56">WhatsApp</a>
  </body>
</html>
```

Version corrigée :

Ta vitrine, avec un lien WhatsApp

Tu construis **ta** page. Fichier : `vitrine/index.html` . En l'ouvrant, ton navigateur est le client.

À avoir pour valider

- ☐ Un fichier nommé `index.html` (pas `.txt`).
- ☐ Un `<h1>` avec ton prénom (ou un pseudo).
- ☐ Un paragraphe : qui tu es, en 2 ou 3 phrases.
- ☐ Une liste : 3 choses que tu veux apprendre cette semaine.
- ☐ Un lien `https://wa.me/...` vers **ton** numéro, texte : Bonjour, je viens de ta vitrine KoffieDev .
- ☐ Au moins 3 règles CSS : fond, texte, bouton.

Si tu as le temps

- Une photo `photo.jpg` dans le même dossier, avec un `alt` .
- Un sous-titre `<h2>` « Me contacter ».

Colle ça, en changeant le numéro

```
https://wa.me/228VOTRENUMERO?text=Bonjour%2C%20je%20viens%20de%20ta%20vitrine%20KoffieDev
```

Exemple : 90 12 34 56 au Togo → 22890123456 .

SI ÇA NE S'AFFICHE PAS

- Le fichier s'appelle `index.html.txt` → affiche les extensions dans l'explorateur Windows.
- Word s'ouvre au lieu de Chrome → clic droit, ouvrir avec le navigateur.
- Accents bizarres → il manque `charset="UTF-8"` .
- WhatsApp n'ouvre rien → enlève le + et les espaces du numéro.
- Tu as modifié le fichier mais l'écran n'a pas bougé → enregistre, puis F5.

Modèle à adapter (prénom + numéro)

Copie uniquement si tu n'as plus de temps. Change Awa et le numéro. C'est *ta* page.

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <title>Vitrine de Awa</title>
    <style>
      body {
        font-family: sans-serif;
        background: #efe6d6;
        color: #1a1612;
        max-width: 640px;
        margin: 40px auto;
        padding: 24px;
        line-height: 1.5;
      }
      h1 { color: #c64520; }
      a.whatsapp {
        display: inline-block;
        background: #167a39;
        color: white;
        text-decoration: none;
        padding: 12px 18px;
        border-radius: 999px;
      }
    </style>
  </head>
  <body>
    <h1>Awa Koffi</h1>
    <p>Je découvre l'informatique avec le bootcamp KoffieDev.
    Aujourd'hui j'ai compris client, serveur, domaine et HTTPS.</p>
    <h2>Cette semaine je veux</h2>
    <ul>
      <li>Mieux comprendre Internet</li>
      <li>Écrire du Python</li>
      <li>Savoir me protéger en ligne</li>
    </ul>
    <p>
      <a class="whatsapp"
        href="https://wa.me/22890123456?
text=Bonjour%2C%20je%20viens%20de%20ta%20vitrine%20KoffieDev">
        Écrire sur WhatsApp
      </a>
    </p>
  </body>
</html>
```

Glossaire du jour

MOT	EN UNE PHRASE
Client	Le programme qui demande (navigateur, app).
Serveur	Le programme qui attend et répond.
Requête / réponse	Le dialogue : le client demande, le serveur envoie.
IP	L'adresse numérique d'une machine.
Nom de domaine	Le nom lisible d'un site.
TLD	L'extension : <code>.com</code> , <code>.ci</code> ...
DNS	L'annuaire qui transforme le nom en IP.
Protocole	Les règles du dialogue sur le réseau.
TCP	Le transport fiable des données.
HTTP	Le protocole d'envoi des pages web (GET, codes 200, 404).
HTTPS	HTTP chiffré, port 443, cadenas.
Port	La « porte » du serveur (80 ou 443 pour le web).
GET	La requête HTTP : « donne-moi cette ressource ».
URL	Protocole + domaine + chemin, en une adresse.
HTML	Le langage de structure d'une page.
CSS	Le langage d'apparence.
404	Le serveur n'a pas ce fichier.

Checklist avant d'envoyer ton fichier

- ☐ Le fichier s'appelle `index.html` .
- ☐ La première ligne est `<!DOCTYPE html>` .
- ☐ Le lien WhatsApp commence par `https://` .
- ☐ Le prénom s'affiche dans le navigateur.
- ☐ Le bouton ouvre une conversation vers toi.
- ☐ Tu as enregistré, puis rechargé avec F5.

MINI-EXO DE FIN DE SÉANCE

Dépose ta vitrine sur MyHub, espace étudiant. Inscription : prénom, nom, numéro de téléphone.
Connexion : prénom et numéro de téléphone.

Tu y télécharges aussi ce support, tu suis ta progression, et tu soumetts le fichier `index.html`.

POUR LE JOUR 2 · PYTHON

Ne jette pas le dossier `vitrine`. On reprend la même page : un script écrira le texte à ta place.
Même heure, même visio, ordinateur encore obligatoire.